

POSSIBILI Q&A ORALE LABORATORIO 3

- 1. implements Runnable vs extends Thread:** Implementare Runnable permette di separare un task dal thread che lo esegue, mentre estendere Thread combina il comportamento del thread con l'operazione da eseguire. Usando Runnable, la classe può estendere altre classi, mentre estendendo Thread, la classe eredita direttamente le capacità del thread, limitando l'ereditarietà a una sola classe.
- 2. start() vs run():** Il metodo start() crea un nuovo thread e invoca il metodo run() su quel thread, mentre chiamare run() direttamente esegue il codice all'interno del thread corrente, senza creare un nuovo thread.
- 3. Thread overhead:** Creare nuovi thread ha un costo in termini di memoria e risorse di sistema, poiché ogni thread ha il proprio stack e richiede un cambio di contesto da parte della CPU, il che può influire sulle performance in presenza di troppi thread.
- 4. Thread Pool:** Gruppo di thread pre-creati e riutilizzabili per gestire attività concorrenti in modo efficiente, così da evitare la continua creazione e distruzione di thread.
- 5. FixedThreadPool vs CachedThreadPool:** FixedThreadPool ha un numero fisso di thread che rimangono attivi fino a quando non vengono terminati. Il numero di thread non cambia, anche se tutti sono occupati. CachedThreadPool crea nuovi thread secondo necessità, ma riutilizza quelli inattivi, utile per eseguire molte attività leggere e intermittenti.
- 6. ThreadPoolExecutor:** È una classe flessibile che gestisce la creazione, gestione e terminazione di thread in un pool. Consente di configurare il numero massimo di thread, il tempo di inattività e la gestione delle attività in coda. I suoi parametri sono: CorePoolSize, MaximumPoolSize, keepAlivetime, workqueue, rejectionhandler.
- 7. Executor lifecycle:** Un Executor ha diversi stati di ciclo di vita, tra cui "running" (attivo), "shutdown" (in fase di spegnimento, non accetta nuovi compiti ma completa quelli attivi) e "terminated" (tutti i compiti sono completati e il pool è chiuso).
- 8. Dimensione thread pool:** La dimensione ottimale del thread pool dipende dal numero di core della CPU e dalla natura dei task. Per task CPU intensive, la dimensione è pari al numero di core della CPU, mentre per task I/O intensive, la dimensione dev'essere pari a un numero alto.
- 9. Scheduled Executor Service:** Esegue un task dopo un ritardo o periodicamente. È utile per operazioni che devono ripetersi regolarmente, come il controllo di risorse o la sincronizzazione.
- 10. Condividere risorse tra thread:** Quando più thread accedono a risorse condivise, si utilizzano meccanismi di sincronizzazione come blocchi synchronized o classi di lock espliciti per prevenire condizioni di gara (race conditions) e garantire consistenza.
- 11. (Array/Linked)BlockingQueue:** Sono code thread-safe che bloccano l'inserimento o l'estrazione di elementi quando la coda è piena o vuota. ArrayBlockingQueue usa un array con dimensione fissa, mentre LinkedBlockingQueue usa una struttura di nodi collegati, permettendo una capacità teoricamente illimitata. Sono utilizzate spesso in scenari di produttore/consumatore.
- 12. Produttore/consumatore:** Modello di programmazione in cui il produttore genera dati e li inserisce in una coda condivisa, mentre il consumatore preleva e processa tali dati. È utilizzato per gestire il flusso di dati tra thread con velocità di produzione e consumo diverse.

13. Lock implicite vs esplicite: Le lock implicite avvengono tramite il costrutto `synchronized` e sono più facili da gestire ma meno flessibili. Le lock esplicite, come quelle fornite dall'interfaccia `Lock`, offrono maggior controllo, come la possibilità di tentare di acquisire una lock con timeout o di rilasciarla manualmente.

14. Stati di un thread:

- **New:** appena creato ma non ancora avviato.
- **Runnable:** pronto per l'esecuzione.
- **Blocked:** in attesa di una risorsa.
- **Waiting/Timed Waiting:** in attesa indefinita o con timeout.
- **Terminated:** ha completato la sua esecuzione.

15. Runnable vs Callable: `Runnable` è un'interfaccia che definisce un singolo metodo `run()` e non restituisce alcun valore. `Callable` invece è più potente poiché il metodo `call()` può restituire un risultato e lanciare eccezioni controllate.

16. Future: È un oggetto che rappresenta il risultato di un task asincrono. Permette di ottenere, una volta completata l'esecuzione, il risultato di un task avviato mediante l'interfaccia `Callable` e fornisce metodi per controllare se l'operazione è terminata o cancellarla.

17. Monitor: È un meccanismo di sincronizzazione che consente di bloccare l'accesso alle risorse condivise tra più thread. In Java, il monitor è associato al costrutto `synchronized`.

18. Meccanismi di coordinazione: Meccanismi come `wait()`, `notify()` e `notifyAll()` permettono ai thread di comunicare tra loro, ad esempio facendo attendere un thread fino a quando una condizione non viene soddisfatta.

19. Stream (Java I/O): Gli stream in Java vengono utilizzati per leggere e scrivere flussi continui di dati da e verso risorse come file, reti o console. Gli `InputStream` e `OutputStream` sono la base per la gestione di byte, mentre `Reader` e `Writer` sono per i caratteri.

20. Filter: I filtri sono stream usati con altri stream per modificare o trasformare i dati man mano che fluiscono attraverso uno stream, ad esempio per cifrare, comprimere o convalidare i dati. Implementano dei buffer.

21. (File/Buffered/Data)Input/OutputStream: Tipi diversi di stream per leggere/scrivere file, buffer o lavorare con tipi primitivi:

- **FileInputStream/FileOutputStream:** leggono e scrivono byte direttamente su file.
- **BufferedInputStream/BufferedOutputStream:** migliorano le prestazioni aggiungendo buffering.
- **DataInputStream/DataOutputStream:** leggono e scrivono dati primitivi (int, float, etc.).

22. Monitor vs lock esplicite: Il monitor usa una sincronizzazione implicita (`synchronized`), mentre le lock esplicite offrono più controllo.

23. Coarse/fine-grain synchronization: La sincronizzazione a grana grossa blocca grandi sezioni di codice o risorse, riducendo il parallelismo. La sincronizzazione a grana fine utilizza lock più piccoli e specifici, permettendo una maggiore concorrenza.

24. Collections: È un framework che fornisce implementazioni di strutture dati comuni come liste, set e mappe, le quali facilitano la gestione di gruppi di dati.

25. Iterators: Sono oggetti che permettono di iterare sugli elementi di una collezione senza esporre i dettagli della sua implementazione. Offrono metodi per navigare tra gli elementi e, in alcuni casi, modificarli.

- 26. Non-thread safe Java collections:** Collezioni standard come ArrayList o HashMap non sono thread-safe, quindi devono essere sincronizzate manualmente se usate in ambienti concorrenti.
- 27. Synchronized collections:** Versioni thread-safe delle collezioni Java, ma meno efficienti delle collezioni concorrenti.
- 28. Classi conditionally thread safe:** Classi che sono thread-safe solo in certe condizioni, ad esempio se usate in modo corretto o se viene garantita una sincronizzazione esterna.
- 29. Blocchi sincronizzati:** Porzioni di codice racchiuse in blocchi synchronized per garantire che solo un thread alla volta possa eseguire quel codice.
- 30. Concurrent collections:** Collezioni progettate per il multithreading, come ConcurrentHashMap, che migliorano le performance rispetto alle collezioni sincronizzate tradizionali.
- 31. Concurrent hash map:** Versione concorrente di HashMap, che utilizza una segmentazione interna per ridurre il contenzioso sui lock e aumentare la scalabilità.
- 32. Operazioni composte atomiche:** Operazioni che devono essere eseguite come un'unità indivisibile, come confrontare e aggiornare valori contemporaneamente, realizzate con classi atomiche o blocchi di lock.
- 33. InetAddress:** È una classe che rappresenta un indirizzo IP. Incapsula in un unico oggetto l'indirizzo IP e il suo nome di dominio. Consente di risolvere nomi di dominio (DNS) o gestire indirizzi IP direttamente.
- 34. Paradigma client/server:** Modello architetturale in cui un client richiede servizi e il server lo fornisce. Il client e il server comunicano attraverso una rete.
- 35. Socket:** È un endpoint per la comunicazione tra due macchine tramite la rete. In Java, la classe Socket permette la comunicazione TCP, mentre DatagramSocket supporta UDP.
- 36. Half closed socket:** È una chiusura parziale di un socket, dove una direzione (input o output) viene chiusa, ma l'altra rimane aperta per continuare la comunicazione nell'altra direzione.
- 37. Server Socket:** Un socket che ascolta su una porta, accetta connessioni in entrata dai client (welcome socket, passivi) e crea un socket per comunicare con ciascun client (connection socket, attivi).
- 38. Try-with-resources:** Costrutto che assicura che le risorse (come stream o file) siano chiuse automaticamente una volta che escono dal blocco, riducendo la possibilità di perdite di risorse.
- 39. Multithreaded server:** Un server che gestisce più client contemporaneamente, creando un nuovo thread per ciascuna richiesta o utilizzando un thread pool.
- 40. Channels (Java NIO):** I canali offrono una gestione I/O più efficiente rispetto agli stream tradizionali, con il supporto per operazioni non bloccanti e multiplexing.
- 41. Canale:** Un canale è una connessione bidirezionale per eseguire operazioni I/O (lettura e scrittura) su dati come file o socket. Alternativa più efficiente rispetto agli stream.
- 42. Buffer:** È una regione di memoria che memorizza temporaneamente i dati letti da un canale prima di elaborarlo o scriverlo.

43. Selettore: È una componente di Java NIO che consente a un singolo thread di monitorare più canali per operazioni di I/O, riducendo la necessità di utilizzare più thread.

44. Interazione JVM-SO: La JVM interagisce con il sistema operativo per la gestione delle risorse di basso livello, come memoria e CPU, e la gestione dei thread.

45. Direct vs non-direct buffer: Un direct buffer interagisce direttamente con la memoria fuori dalla JVM, rendendo più veloce l'I/O, mentre un non-direct buffer è gestito nella memoria heap della JVM.

46. Scrittura/lettura canale: Scrivere o leggere da un canale implica l'uso di buffer per gestire i dati, permettendo operazioni più efficienti rispetto agli stream tradizionali. Non si scrive/legge direttamente su un canale, ma si passa attraverso un buffer.

47. (File/Datagram/Socket/ServerSocket)Channel: Sono implementazioni di Channel per gestire file, pacchetti UDP, socket TCP e connessioni server.

- **FileChannel:** legge e scrive dati da/verso file.
- **DatagramChannel:** mediante una connessione UDP, permette l'invio e la ricezione di pacchetti.
- **SocketChannel:** mediante una connessione TCP, permette di connettersi a un server TCP e leggere/scrivere dati in modalità non bloccante.
- **ServerSocketChannel:** accetta connessioni TCP da client e crea un SocketChannel per la gestione la comunicazione con ogni client.

48. Trasferimento diretto tra canali: Operazione che permette di trasferire dati direttamente da un canale all'altro senza passare per un buffer intermedio.

49. Non-blocking connect: Tentativo di stabilire una connessione senza bloccare il thread chiamante.

50. Multiplexing: Consente a un thread di gestire più connessioni I/O simultaneamente attraverso i selettori, migliorando l'efficienza dell'uso dei thread.

51. Selector: Monitora molteplici canali e notifica quando uno di essi è pronto per un'operazione di I/O (lettura, scrittura, connessione).

52. Registrazione canale su selettore: Perché un canale sia monitorato da un selettore, deve essere registrato con una SelectionKey, che associa il canale con le operazioni da monitorare.

53. Interest set: Set di eventi a cui un canale è interessato, come operazioni di lettura o scrittura.

54. SelectionKey: Oggetto che rappresenta l'associazione tra un canale e un selettore. Memorizza il set di interessi e l'allegato e contiene informazioni su quali operazioni sono pronte su quel canale.

55. Select: Operazione che blocca il thread finché almeno uno dei canali registrati non è pronto per un'operazione.

56. ReadySet: È l'insieme di SelectionKey che rappresentano i canali pronti per le operazioni I/O.

57. (Selected/Cancelled)Key Set:

- **SelectedKey Set:** Contiene le SelectionKey corrispondenti ai canali che sono stati selezionati (ready) dal selettore, ovvero sono pronti per le operazioni di I/O.
- **CancelledKey Set:** Contiene le SelectionKey corrispondenti ai canali che sono stati cancellati e che non verranno più monitorate dal selettore.

- 58. Attachment della SelectionKey:** Oggetto che può essere associato a una SelectionKey per fornire ulteriori informazioni sull'operazione di I/O.
- 59. TCP vs UDP:** TCP è un protocollo orientato alla connessione e garantisce la consegna affidabile dei dati, mentre UDP è senza connessione, veloce ma meno affidabile.
- 60. Costruttori DatagramPacket:** Costruttori per creare pacchetti UDP con un array di byte, la lunghezza e l'indirizzo di destinazione.
- 61. DatagramPacket:** Rappresenta un pacchetto di dati usato nella comunicazione UDP. Contiene sia i dati che l'indirizzo di destinazione.
- 62. ByteArrayInputStream/OutputStream:** Stream che leggono o scrivono dati direttamente in un array di byte.
- 63. UDP e serializzazione:** In UDP, gli oggetti possono essere convertiti in byte tramite serializzazione e inviati come pacchetti.
- 64. Socket UDP:** Endpoint che invia e riceve pacchetti di dati senza stabilire una connessione tra client e server.
- 65. Multicasting:** Tecnica che permette l'invio di un messaggio a più destinatari in un gruppo.
- 66. Gruppi multicast:** Gruppi di destinatari che ricevono pacchetti inviati tramite multicast.
- 67. API Multicast:** L'API Java fornisce classi per inviare pacchetti multicast, consentendo di trasmettere dati a più destinatari contemporaneamente.
- 68. Socket Multicast:** Socket speciali che consentono di unirsi a un gruppo multicast per ricevere messaggi trasmessi dal gruppo.
- 69. Serializzazione:** Processo di conversione di un oggetto Java in un flusso di byte per memorizzarlo o trasmetterlo.
- 70. JSON:** Formato leggero e leggibile per la rappresentazione di dati strutturati. Spesso usato per la comunicazione tra applicazioni web. JSON è comunemente usato come formato per i dati scambiati nelle API REST su HTTP.
- 71. GSON:** Libreria Google che semplifica la conversione tra oggetti Java e JSON.
- 72. Deserializzazione:** Il processo inverso della serializzazione: converte un flusso di byte in un oggetto Java.
- 73. Reflection:** Meccanismo Java che consente di esaminare o modificare il comportamento dei programmi durante l'esecuzione.
- 74. Serializzazione Java:** Meccanismo nativo in Java per serializzare e deserializzare oggetti. Richiede che gli oggetti implementino l'interfaccia Serializable.
- 75. Serializable:** Interfaccia marker in Java che indica che un oggetto può essere serializzato.
- 76. Externizable:** Interfaccia Java che permette di controllare il processo di serializzazione implementando manualmente metodi di lettura e scrittura.
- 77. Deserializzazione Java:** Ripristina un oggetto Java, precedentemente serializzato, da un flusso di byte.

78. Oggetti non serializzabili: Gli oggetti che non implementano l'interfaccia Serializable e non sono transient non possono essere serializzati.

79. Transient: Modificatore che esclude i campi dalla serializzazione.

80. Volatile: Parola chiave che garantisce che le modifiche a una variabile siano visibili immediatamente a tutti i thread, evitando la memorizzazione nella cache locale del thread.

81. Atomic: Fornisce operazioni atomiche (non interrotte da altri thread) su variabili, utili per evitare condizioni di competizione. Esempi: AtomicInteger, AtomicBoolean.

82. Collezioni e iteratori: Le collezioni Java offrono metodi per memorizzare, organizzare e iterare su insiemi di oggetti, utilizzando iteratori per attraversarle.

83. Iteratori fail-fast/fail-safe/weakly consistent:

- **Fail-fast:** lanciano eccezioni se la collezione viene modificata durante l'iterazione.
- **Fail-safe:** permettono modifiche durante l'iterazione senza sollevare eccezioni, dato che lavorano su una copia della collezione.
- **Weakly consistent:** riflettono alcune ma non tutte le modifiche apportate alla collezione durante l'iterazione, garantendo una coerenza debole.

84. CopyOnWriteArrayList: Implementazione thread-safe di ArrayList, che crea una copia dell'array originale per ogni scrittura, rendendola immutabile durante le iterazioni. Ideale per collezioni che vengono lette spesso ma modificate raramente.

85. ConcurrentHashMap: Implementazione thread-safe di HashMap che utilizza la segmentazione interna per minimizzare i blocchi e limitare la concorrenza sui lock.

EXTRA

1. Quali tipi di iteratori ci sono, e quali sono le differenze?

- **Fail-fast iterator:** Lancia un'eccezione (ConcurrentModificationException) se la collezione viene modificata mentre si sta iterando. Sono usati dalle collezioni come ArrayList o HashMap.
- **Fail-safe iterator:** Permettono la modifica della collezione durante l'iterazione senza lanciare eccezioni, ma non riflettono le modifiche nella stessa iterazione. Usati in collezioni concorrenti come ConcurrentHashMap.
- **Weakly consistent iterator:** Forniscono una vista debole e non consistente dei cambiamenti fatti durante l'iterazione. Gli aggiornamenti non sono garantiti ma non lanciano eccezioni. Usati da collezioni come CopyOnWriteArrayList.

2. Che iteratori usano il wrapper synchronized, copyOnWriteArrayList, ConcurrentHashMap?

- **Wrapper synchronized:** Usa iteratori fail-fast, poiché non sono pensati per scenari concorrenti.
- **CopyOnWriteArrayList:** Utilizza iteratori fail-safe. Questi iteratori lavorano su una copia immutabile della lista, quindi possono iterare senza preoccuparsi delle modifiche.
- **ConcurrentHashMap:** Utilizza iteratori weakly consistent. Consentono modifiche durante l'iterazione, ma i cambiamenti potrebbero non essere immediatamente visibili o riflessi nell'iterazione corrente.

3. Quali sono i parametri di un buffer (position, limit, mark, capacity)?

- **position:** Indica la posizione corrente del buffer, cioè il prossimo indice dove verrà letto o scritto.
- **limit:** Rappresenta il numero massimo di elementi che possono essere letti o scritti nel buffer.
- **mark:** Segna una posizione specifica nel buffer. Può essere ripristinata tramite reset().
- **capacity:** È la dimensione massima del buffer, cioè il numero totale di elementi che può contenere.

4. Come funzionano le operazioni di rewind, flip ecc. di un buffer? Quali sono le differenze tra di loro?

- **rewind():** Imposta la position a 0 e mantiene il limit invariato. È usato per rileggere i dati dal buffer.
- **flip():** Cambia il limit alla posizione corrente e imposta la position a 0. Serve per passare dalla scrittura alla lettura del buffer.
- **clear():** Imposta position a 0 e limit alla capacità totale del buffer, preparandolo per la scrittura.
- **mark() e reset():** mark() salva la position attuale. Con reset() è possibile riportare la position al valore salvato.

Differenze:

- rewind() riporta solo la posizione a 0 per rileggere i dati.
- flip() è usato per passare dalla scrittura alla lettura, regolando sia position che limit.
- clear() prepara il buffer per la riscrittura totale.

5. Come funziona il monitor?

Il monitor è un meccanismo di sincronizzazione che garantisce che solo un thread alla volta possa eseguire un blocco di codice sincronizzato su un oggetto. Ogni oggetto in Java ha un monitor che controlla l'accesso ai suoi blocchi di codice synchronized. Quando un thread entra in un metodo sincronizzato, acquisisce il monitor, e lo rilascia quando esce, consentendo ad altri thread di entrare.

6. Cosa fanno i metodi wait, notify, notifyAll?

- **wait():** Fa attendere il thread corrente finché un altro thread non invoca notify() o notifyAll() sullo stesso oggetto.
- **notify():** Risveglia un singolo thread in attesa su quel monitor.
- **notifyAll():** Risveglia tutti i thread in attesa su quel monitor. Tuttavia, solo uno di essi potrà ottenere il monitor e procedere.

7. Perché ci sono due code associate alla lock di un oggetto?(Entry set, wait set)

- **Entry set:** Contiene i thread che aspettano di ottenere la lock per accedere a un blocco sincronizzato.
- **Wait set:** Contiene i thread che hanno rilasciato temporaneamente la lock tramite wait() e aspettano un segnale da notify() o notifyAll().

8. Cosa sono le variabili volatili? A cosa servono?

Una variabile volatile garantisce che le modifiche fatte da un thread siano immediatamente visibili agli altri thread. È usata per evitare problemi di visibilità dei dati nella memoria quando i thread leggono e scrivono una variabile.

9. Descrivi un esempio di utilizzo di una variabile volatile.

```
private volatile boolean isActive = true;

public void run() {
    while (isActive) {
        // codice
    }
}

public void stop() {
    isActive = false;
}
```

In questo esempio, volatile garantisce che quando isActive viene impostato a false, il cambiamento sia visibile al thread che sta eseguendo il ciclo.

10. Che metodi offrono le concurrent collection?

Le collezioni concorrenti come ConcurrentHashMap offrono metodi aggiuntivi che permettono operazioni sicure in ambienti multi-thread, come:

- **putIfAbsent()**: Aggiunge un valore solo se la chiave non è già presente.
- **remove()**: Rimuove un elemento solo se associato a un determinato valore.
- **replace()**: Sostituisce il valore associato a una chiave solo se ha un valore specificato.

11. Quali sono le differenze tra ArrayBlockingQueue e LinkedBlockingQueue?

- **ArrayBlockingQueue**: Implementa una coda bloccante basata su un array di dimensione fissa. È più veloce ma ha una capacità prefissata.
- **LinkedBlockingQueue**: È basata su una lista concatenata e può avere una dimensione variabile, sebbene possa anche essere limitata. È più flessibile ma introduce un leggero overhead.

12. Quali sono i parametri di una threadPool e a cosa servono?

- **Core pool size**: Numero minimo di thread che il pool mantiene sempre in esecuzione, anche se sono inattivi.
- **Maximum pool size**: Numero massimo di thread che possono essere attivi contemporaneamente.
- **Keep alive time**: Tempo che un thread inattivo rimane nel pool prima di essere terminato, se il numero di thread supera il core pool size.
- **Work queue**: Coda in cui vengono inseriti i task quando non ci sono thread disponibili.

13. Descrivi la CachedThreadPool.

La CachedThreadPool crea nuovi thread secondo necessità e riutilizza i thread inattivi. Non ha un limite sul numero di thread, e i thread inattivi vengono eliminati se non utilizzati per 60 secondi. È utile per eseguire molti task brevi.

14. Perché si dovrebbe usare un blocco synchronized invece che un metodo synchronized?

Un **blocco synchronized** permette di sincronizzare solo una parte specifica di un metodo, mentre un **metodo synchronized** sincronizza l'intero metodo. Usare un blocco synchronized può essere più efficiente, poiché limita la sincronizzazione solo alle sezioni critiche del codice, lasciando il resto del metodo libero da lock.